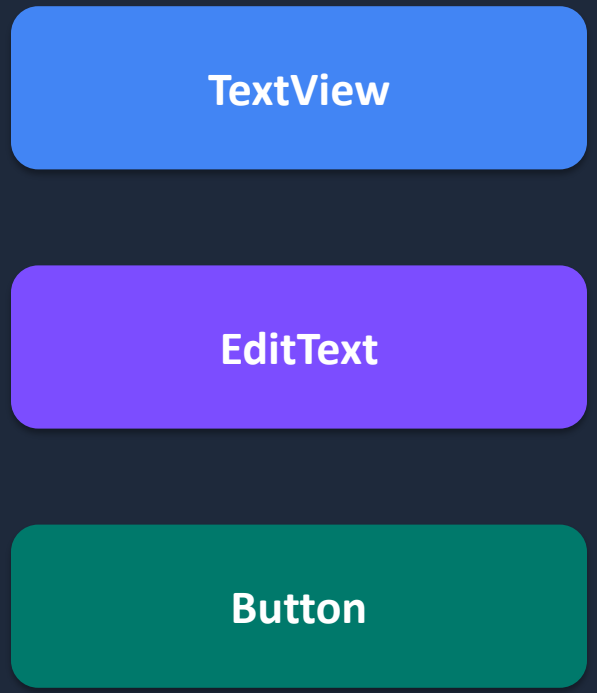
Primeros layouts en Android Studio

De View y ViewGroup a LinearLayout



Objetivos de la sesión

Al finalizar, el alumnado debería poder construir y razonar una interfaz sencilla.

1 · Comprender

View, ViewGroup y jerarquía de vistas

2 · Construir

LinearLayout vertical y horizontal

3 · Controlar

Tamaño, espacio, alineación y peso

4 · Combinar

Layouts anidados y componentes básicos

5 · Resolver

Una interfaz completa sin programar lógica

1. ¿Cómo construye Android una pantalla?

La interfaz no es una imagen: es un árbol de objetos.

View

Elemento de interfaz: texto, botón, imagen, campo…

ViewGroup

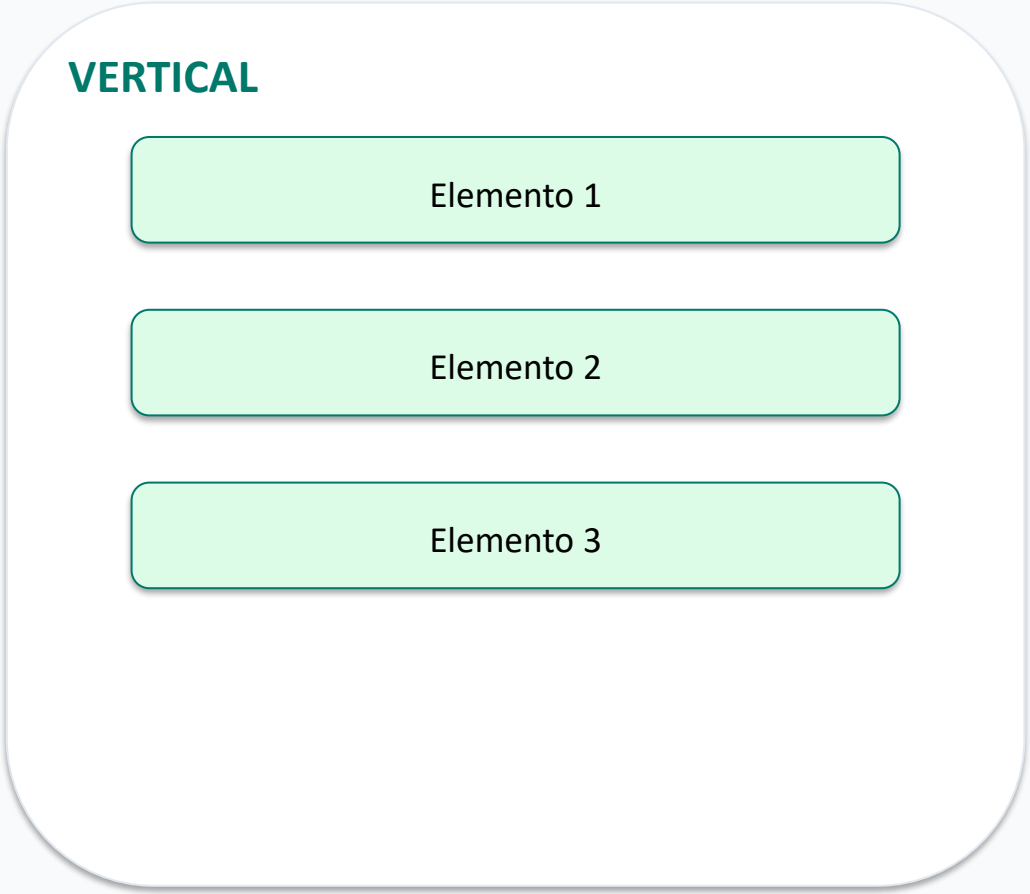
Contenedor que organiza otras Views.

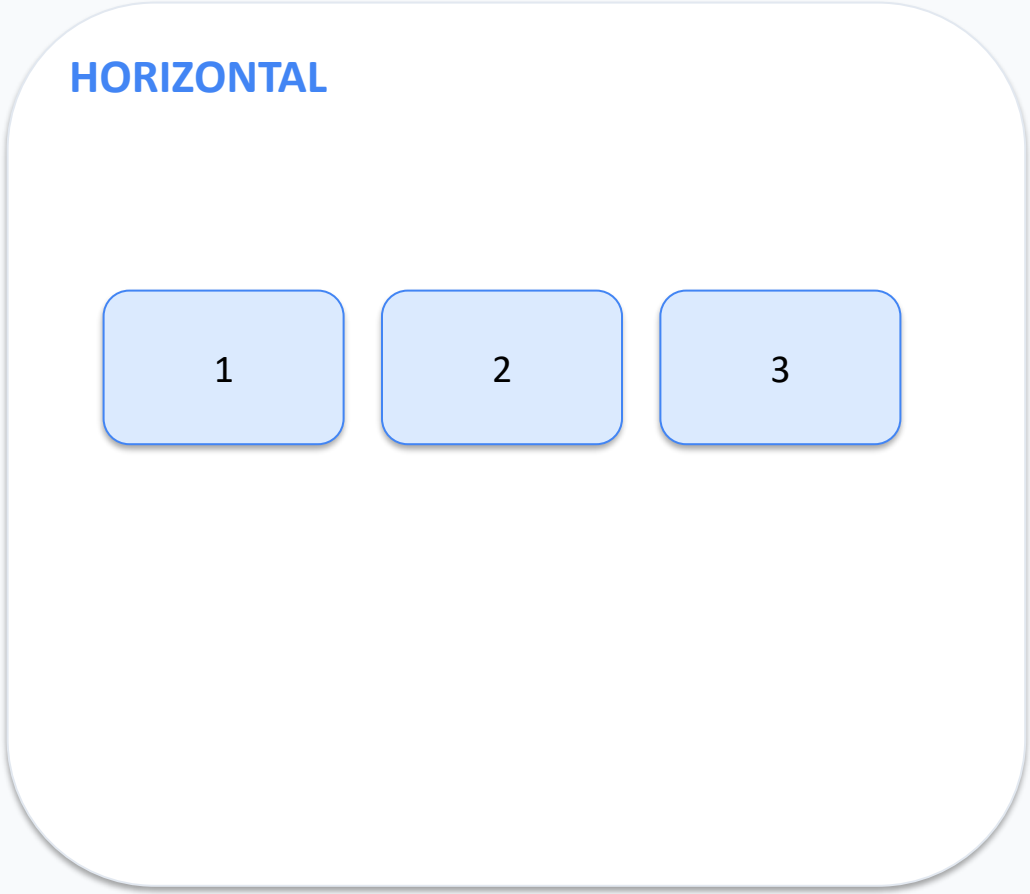
Árbol de vistas

```
LinearLayout   ← ViewGroup
│
├── TextView    ← View
├── EditText    ← View
└── Button      ← View
```

2. LinearLayout: una idea muy simple

Coloca elementos uno detrás de otro.





3. Nuestro primer LinearLayout

Primero identificamos el contenedor; después añadimos Views.

```
<LinearLayout

xmlns:android="http://schemas.android.com/apk/res/and
roid"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

</LinearLayout>
```

orientation define la dirección.

width y height indican cuánto espacio ocupa.

El LinearLayout será el padre de los elementos interiores.

4. Añadimos una View: TextView

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Mi primera aplicación"
    android:textSize="24sp" />
```

Pregunta clave

¿Cuánto espacio quiere ocupar esta View?

Dos dimensiones

layout_width
layout_height

5. wrap_content vs match_parent

Una de las ideas que más se repetirá en Android.

wrap_content

Ocupa solo lo que necesita el contenido.



match_parent

Ocupa todo el espacio que le permite su padre.



6. Construimos un formulario

Título + usuario + contraseña + botón.

```
<TextView ... />

<EditText
    android:hint="Usuario" ... />

<EditText
    android:hint="Contraseña"
    android:inputType="textPassword" ... />

<Button
    android:text="Entrar" ... />
```

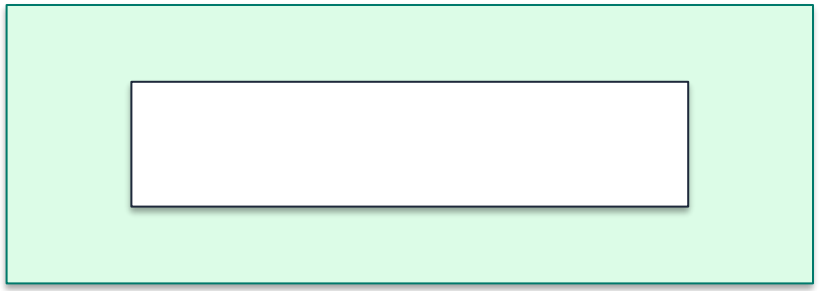


7. Espaciado: padding y margin

La diferencia se entiende mejor visualmente.

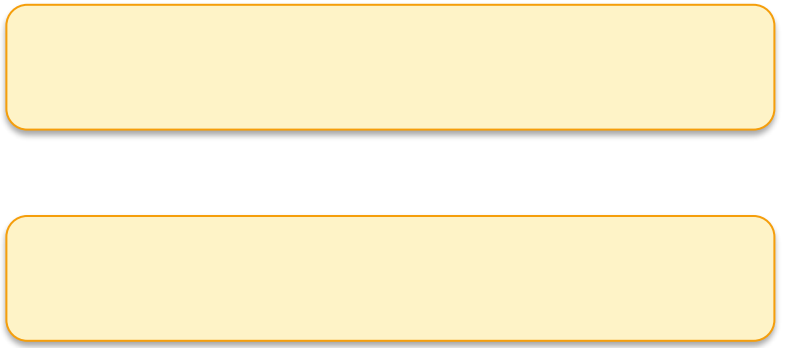
PADDING

Espacio interior: entre el borde y el contenido.



MARGIN

Espacio exterior: separa una View de otras.



8. Unidades básicas: dp y sp

dp

Para tamaños y distancias de interfaz.

Ej.: padding, margin, ancho, alto.

sp

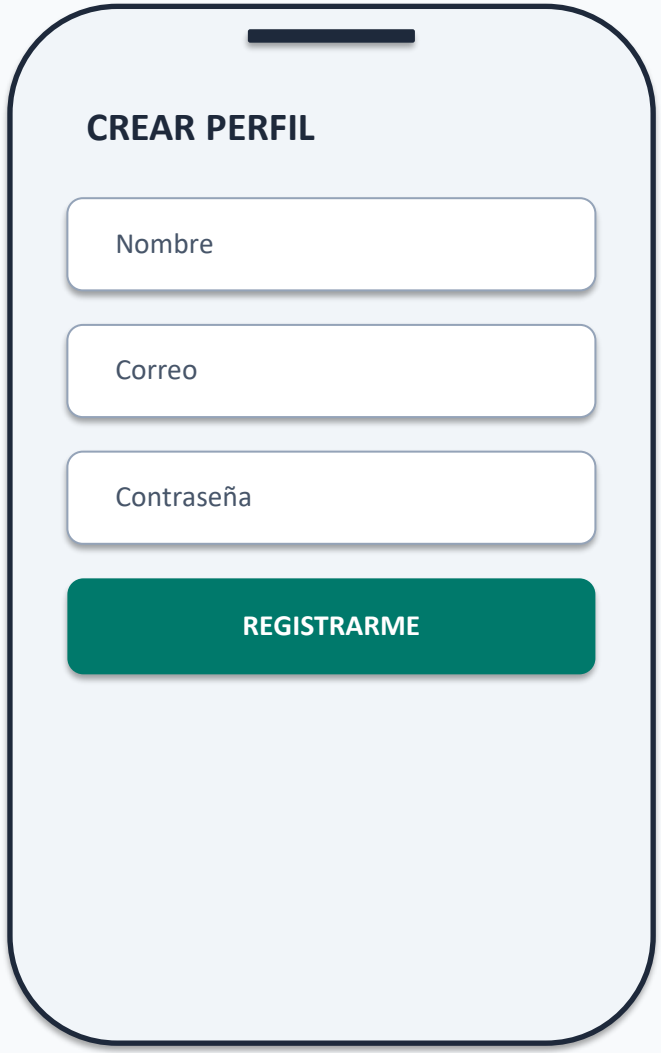
Para tamaños de texto.

Ej.: android:textSize="24sp"

En esta primera sesión: recordar la regla práctica, sin entrar todavía en densidades.

9. Ejercicio guiado: Crear perfil

Ahora trabajan los alumnos.



LinearLayout vertical

padding general de 24dp

1 TextView + 3 EditText + 1 Button

separación con margin

No entregar el XML: deben construirlo

10. LinearLayout horizontal y layouts anidados

```
LinearLayout VERTICAL
│
├── TextView
├── EditText
├── EditText
└── LinearLayout HORIZONTAL
      ├── Button CANCELAR
      └── Button ENTRAR
```

Idea esencial

Un layout puede contener otros layouts.



11. gravity y layout_gravity

Dos preguntas diferentes.

gravity

¿Dónde coloco MI contenido?

Ej.: centrar el texto o los hijos dentro de un contenedor.

layout_gravity

¿Dónde me coloco YO respecto a mi padre?

Conviene probarlo en directo con varios valores.

12. Repartir espacio con layout_weight

```
<Button
    android:layout_width="0dp"
    android:layout_weight="1" ... />

<Button
    android:layout_width="0dp"
    android:layout_weight="1" ... />
```

weight 1 + weight 1

50 % | 50 %

weight 1 + weight 2

1/3 | 2/3

Experimenta

1–3 · 2–2 · 1–4

13. Mini reto: pensar antes de escribir XML

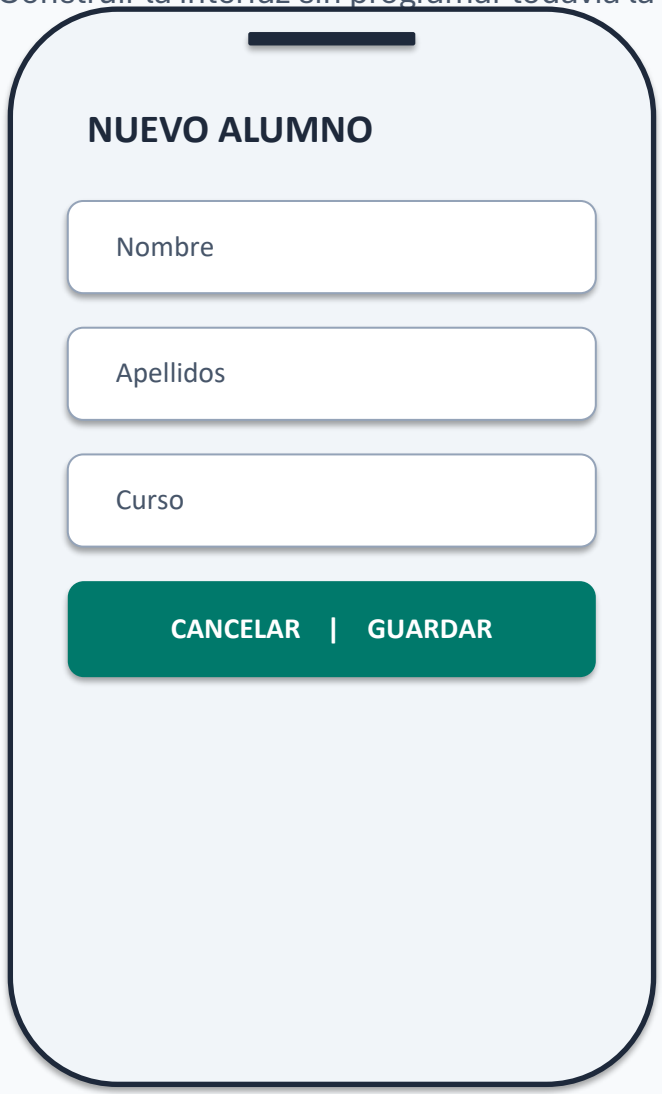
Dibujad primero el árbol de layouts de una calculadora.

```
LinearLayout VERTICAL
│
├── TextView / pantalla
├── LinearLayout HORIZONTAL → 7 · 8 · 9
├── LinearLayout HORIZONTAL → 4 · 5 · 6
├── LinearLayout HORIZONTAL → 1 · 2 · 3
└── Button BORRAR
```



14. Ejercicio final: Nuevo alumno

Construir la interfaz sin programar todavía la lógica.



Usar LinearLayout y componentes básicos.

Aplicar orientation, width y height.

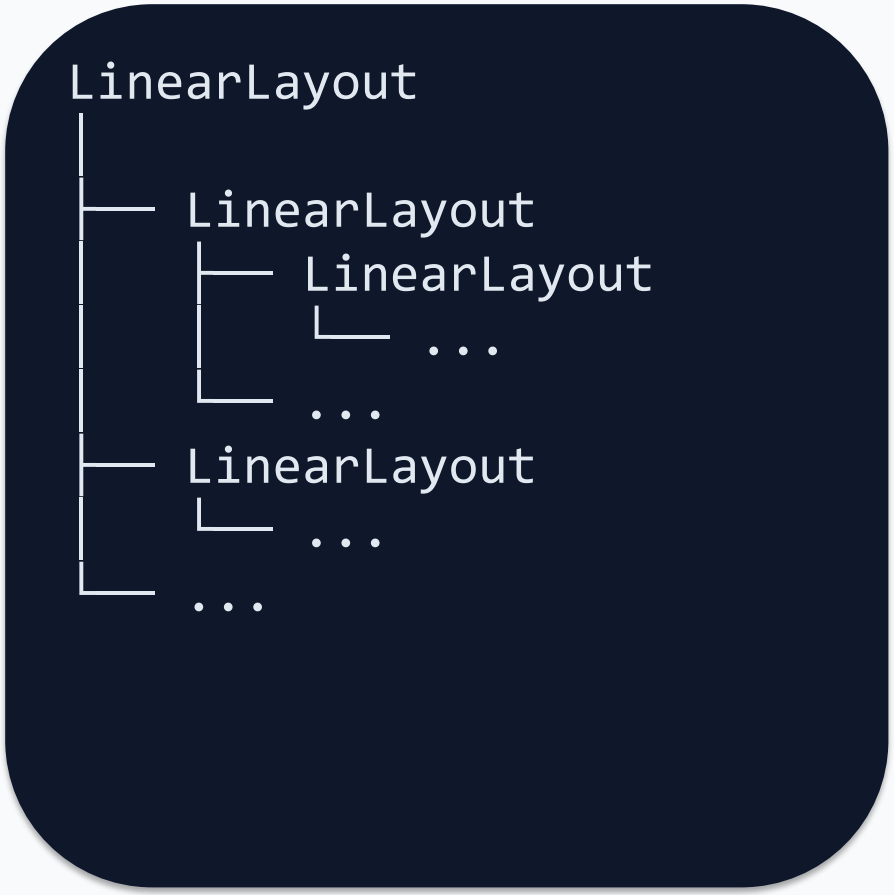
Incluir padding y margin.

Resolver alineación y reparto de espacio.

Objetivo: que se parezca al diseño y puedan explicar su árbol.

15. ¿Qué ocurre cuando la pantalla crece?

LinearLayout funciona, pero los anidamientos pueden multiplicarse.



Pregunta

¿Podemos construirlo así? Sí.

Segunda pregunta

¿Es siempre la solución más cómoda? No.

Puente a la siguiente sesión

Necesitamos relaciones más flexibles entre elementos.

Siguiente paso: ConstraintLayout

“Este botón debajo de este texto.”
“Esta imagen alineada con este elemento.”
“Este campo entre estos dos elementos.”

Primero entendemos el problema → después aprendemos la herramienta